REWIND

Product Requirements Document

AI-powered story and language companion for Fire TV

An AI companion that helps viewers catch up on a story, understand unfamiliar characters, and get spoiler-safe explanations in language that suits them.

Challenge	Amazon App Development Challenge 2026
Track	Fire TV
Version	1.0 — MVP PRD
Delivery window	5–6 days
Primary audience	Everyday consumers and entertainment viewers

PRODUCT VISION

Make every story easier to understand, regardless of when you start watching or which language you speak.

Core principle: build a complete, reliable TV-native story-understanding journey before adding advanced streaming or live-audio capabilities.

1. Executive Summary & Problem Statement

Executive summary. Rewind is an AI-powered companion for Fire TV that helps viewers understand movies and series through spoiler-aware recaps, character explanations, and language assistance. Users can ask questions about a story, explore character relationships, and receive explanations tailored to their preferred language and comprehension level.

Problem statement. Viewers forget plot details, struggle to understand unfamiliar characters, or miss context when watching international content. Searching externally interrupts viewing and can expose viewers to unwanted spoilers. Language barriers make international films harder to follow.

Value proposition. Rewind brings contextual story understanding and accessible language assistance directly to the television experience, reducing the need to leave the screen to understand what is happening.

Product goals

- Deliver an end-to-end Fire TV-compatible application.
- Demonstrate accurate, context-grounded, spoiler-aware AI explanations.
- Support a limited, demonstrable translation workflow.
- Provide an accessible interface navigable with a TV remote.

2. User Personas & User Journey

Primary persona: The Curious Viewer

Attribute	Description
Goal	Understand a story without interrupting the viewing experience
Pain points	Forgotten plot details, confusing characters, language barriers, accidental spoilers
Environment	Television, remote control, limited text input
Success outcome	Understands a story or dialogue without leaving Rewind

Core user journey

Step	User action	System response
1	Open Rewind	Browse a small catalog of supported stories
2	Select a story	Choose a movie, episode, or available transcript
3	Choose an action	Catch Me Up, Explain a Character, or Translate
4	Request assistance	Receive a context-grounded recap, explanation, or translated dialogue
5	Explore further	Ask a follow-up question or return to the story

MVP constraint: Users select the movie and watched position manually. Automatic recognition of currently playing content is not required.

3. MVP Scope & Feature Prioritization

Priority	Features	Acceptance criteria
MUST	Fire TV-compatible interface	Primary actions work with D-pad and select button
MUST	Story catalog	At least 2 demo stories with structured episode and character metadata
MUST	AI recap and character Q&A;	Concise answer grounded in selected story context
MUST	Spoiler-aware responses	Responses respect selected watched episode and avoid intentional future-story disclosure
MUST	Language assistance	Translate supplied transcript or subtitle text into English or Hindi
MUST	Loading, error, and fallback states	App remains usable when AI fails or returns invalid output
SHOULD	Character relationship explorer	Displays character connections and short explanations
SHOULD	Explanation-level selector	Offers simple and detailed explanations
SHOULD	Persistent preferences	Saves language and preferences using Supabase
OUT	Third-party streaming integrations; live audio capture; synchronized live subtitles; custom-trained models; complex vector search; accounts/social/payments	Not required for the MVP; revisit only after feasibility validation

Scope rule: If the core recap and Q&A; flow is not stable by Day 3, defer all should-have features until it works end to end.

4. Functional Requirements

FR-01 — Browse and select a story

- The user opens Rewind; the system displays demo stories and supported actions.
- The user selects a story using the remote; the system loads story metadata, available episodes, and supported languages.
- Acceptance: Selection works without a keyboard or mouse; unsupported content is clearly identified.

FR-02 — Catch Me Up

- The user selects a story and last-watched episode, then chooses Catch Me Up.
- The system retrieves permitted summaries and relevant character information.
- The AI generates a concise recap, displayed with an option to request more detail.
- Acceptance: The recap uses the selected story context and does not intentionally reveal events beyond the chosen watch position.

FR-03 — Explain a character

- The user chooses Explain a Character and selects a character or enters a question.
- The system retrieves biography, relationships, and relevant events within the spoiler boundary.
- The AI responds in the selected language and comprehension level.
- Acceptance: Answers are grounded in available metadata; missing information is acknowledged rather than invented.

FR-04 — Translate dialogue

- The user selects Translate, chooses a supplied transcript or subtitle sample, and selects English or Hindi.
- The system sends the text to the translation service and displays the translated dialogue.
- Acceptance: Meaning is preserved, failures are handled, and the UI makes clear that supplied text is being translated—not audio from another streaming app.

FR-05 — Spoiler protection

- The user selects the last episode or story position watched.
- The backend filters story context before calling the AI.
- The system validates the response against the permitted scope and blocks or regenerates responses that fail validation.
- Acceptance: Predefined future-plot leakage tests pass; prompt instructions alone are not treated as sufficient protection.

FR-06 — Error handling and security

- The system displays loading feedback during requests.
- On failure, it offers retry or a prepared fallback response.
- The backend validates input length, language, and story identifiers; credentials remain server-side.
- Acceptance: Invalid requests are rejected, credentials are not exposed in client code, and loading states cannot hang indefinitely.

5. Technical Architecture & Constraints

Recommended technology stack

Layer	Technology	Purpose
TV frontend	React Native with a Fire TV-compatible starter	Remote-first user interface
Alternative frontend	React web app packaged for a verified compatible runtime	Use only if deployment compatibility is proven early
Backend/API	Vercel Functions + TypeScript	Request handling and AI orchestration
AI reasoning	xAI Grok API	Recaps, character explanations, contextual Q&A;
Database	Supabase Postgres	Stories, characters, episode summaries, optional preferences
Translation	WhisperLabs, subject to API verification	Use only if the selected service supports the required endpoints
Version control	GitHub	Collaboration, source code, setup instructions

Platform requirement: A website deployed to Vercel alone does not satisfy the Fire TV requirement. Verify the app on Fire OS or Vega OS, or an officially supported simulator. Confirm the chosen WhisperLabs product and API before implementation; speech-to-text does not automatically imply translation support.

High-level data flow

1. Fire TV frontend	Story selection, question, watch position, target language
2. Vercel API	Validate request, select workflow, enforce input limits
3. Context/service layer	Fetch permitted story context from Supabase, or send supplied dialogue to translation service
4. AI + validation	Grok generates grounded explanations; backend applies output limits and spoiler checks
5. TV response	Display recap, explanation, or translated dialogue

Suggested data model

- stories: story ID, title, description.
- episodes: episode ID, story ID, sequence number, summary.
- characters: character ID, story ID, name, biography.
- character_relationships: character IDs, relationship description, relevant episode.
- story_facts: fact, source/reference, earliest episode where the fact is revealed.
- user_preferences (optional): preferred language and explanation level.

Engineering constraints

- TV compatibility: Verify a working build on Day 1; use visible focus states, large text, and D-pad navigation.
- Latency: Keep answers short, show loading feedback, and use request timeouts.
- Spoiler safety: Combine episode-scoped retrieval with response validation.
- API costs: Use a small catalog, bounded prompts, and limited AI calls.
- Credentials: Keep Grok, Supabase service-role, and translation secrets in server-side environment variables.
- Privacy: Avoid collecting viewing history or personal data beyond MVP needs.
- Fallbacks: Prepare sample recaps and explanations so temporary API failures do not derail the demo.

6. Success Metrics & Judging Alignment

MVP success metrics

Metric	Target for demo readiness
End-to-end functionality	All four core flows work: story selection, recap, character Q&A;, and text translation
Response quality	At least 8 of 10 predefined story questions receive factually supported answers
Spoiler protection	All predefined future-plot leakage tests pass
Remote usability	All primary actions can be completed using the TV remote
Reliability	Core flows succeed across 10 consecutive scripted demo runs
Translation quality	At least 8 of 10 reviewed translations preserve intended meaning
Demo readiness	Runs on a supported Fire TV device or simulator, with a tested AI-failure fallback

Judging alignment

Criterion	How Rewind addresses it
Innovation	Combines story understanding, character context, spoiler-aware responses, and language assistance in one TV-native experience
Technical implementation	Integrates an LLM, structured story retrieval, translation API, backend orchestration, and a Fire TV-compatible frontend
Design and UX	Remote-first navigation, readable text, focused interactions, and short TV-friendly explanations
Potential impact	Helps viewers overcome language barriers and understand unfamiliar or complex stories without constant external searching
Quality of idea	Solves a recognizable consumer problem with an end-to-end demonstration rather than a generic chatbot

5–6 day execution plan

Day	Deliverable
1	Set up Fire TV-compatible starter, D-pad navigation, and verify target environment.
2	Build catalog, story/episode selection, and recap screens using seeded data.
3	Integrate Grok for recaps and character Q&A; add context filtering and fallbacks.
4	Implement spoiler controls and translate supplied transcript/subtitle text.
5	Test remote navigation, grounding, spoiler boundaries, errors, and demo journey.
6	Fix critical defects, record demo, finalize README, verify submission requirements.

PRODUCT DECISION

Build Rewind as a story-understanding companion first, with translation as its second capability. Do not attempt automatic access to audio from third-party streaming apps or real-time translation of arbitrary movies within this MVP.

This scope is achievable while preserving the product's strongest differentiator: helping viewers understand what they watch, in a language and level of detail that works for them.